Grupa: 131 Numele şi prenumele (cu MAJUSCULE): _____

Arhitectura Sistemelor de Calcul: Parţial - 10% din nota finală

Exerciţiul 1 [1.5p] *Scrieţi dimensiunea alocată în RAM, atât în biţi, cât şi în Bytes, pentru următoarele variabile declarate în secţiunea* `.data`*:*

Declarare în `.data`	dimensiune în biţi	dimensiune în Bytes
`str1: .ascii "Partial ASC"`		
`mySpace: .space 400`		
`myWord: .word 2222`		
`myLong: .long 0x7f333025`		
`myByte: .byte 'b'`		

Scrieţi un apel de sistem `WRITE` (codul apelului este 4) pentru variabila `str1`, astfel încât să afişaţi, la `standard output`, întreg conţinutul şirului de caractere `str1` şi nimic altceva în plus.

Exerciţiul 2 [1p] *Completaţi următorul tabel, cu valorile corecte în bazele 2 şi 4.*

Baza 2	Baza 4
	20001003000100
1110 0101 0101 1000 0100 1000 1110 0011	

Exerciţiul 3 [2p] *Fie următorul program, scris în* `assembly` *(procesor Intel x86, sintaxa AT&T):*

```
.data
    x: .long 0x10002
    z: .long 0x0000022
    t: .long 0x20
.text
.global main
main:
    movl x, %eax
    mul z
    jmp et2
et1:
    jmp et_exit
    divl t
    decl %eax
et2:
    jmp et1
et_exit:
    movl $1, %eax
    xorl %ebx, %ebx
    int $0x80
```

Executăm, în `debugger`*, următoarele comenzi:*

```
b main; b et_exit; run; stepi; i r eax; stepi; c; i r eax; stepi; stepi; i r ebx
```

Valorile pe care le vom obţine sunt, în ordine:

EAX (1)	EBX	EAX (2)

Important! *Completaţi valorile în baza 10, scrise ca sume de puteri ale lui 2.*

Exerciţiul 4 [1.5p] *Fie următoarele instrucţiuni în* `main`*:*

```
movl $0x100002, %eax
movl $0x0020, %ebx

mul %ebx
div p
```

Descrieţi ce obţinem dacă $p = 16$. Dar dacă $p = 8$?

Exerciţiul 5 [1p] *Fie următorul program, scris în* **assembly** *(procesor* **Intel x86**, *sintaxa* **AT&T**):

```
.data
    x: .long 58
    y: .long 26
.text
.global main
main:
    movl x, %eax
    subl y, %eax
    xorl %edx, %edx
    movl $7, %ecx
    divl %ecx
    addl %edx, %eax
    mul %eax
    cmp $131, %eax
et1:
    movl $2, %ebx
et2:
    movl $0, %ebx
et_exit:
    mov $1, %eax
    xorl %ebx, %ebx
    int $0x80
```

Completaţi valorile regiştrilor în momentul în care executarea programului ajunge în dreptul etichetei **et_exit** *(după ce am rulat în* **debugger** *comenzile* **b et_exit; run; i r**):

EAX	EBX	ECX

Exerciţiul 6 [1p] *Fie următorul program, dezvoltat în limbajul de asamblare al procesorului* **Intel x86**, *sintaxa* **AT&T**:

```
.data
.text
.global main
main:
    movl $0xb03f11a0, %eax
    mov $0, %ebx
    xorl %ecx, %ecx
    movw %ax, %bx
    movb %bl, %al
    movb %ah, %ch
    mov $3, %ecx
et_loop:
    addb %al, %bl
    loop et_loop
et_exit:
    movl $1, %eax
    xorl %ebx, %ebx
    int $0x80
```

Valoarea stocată în dreptul etichetei **et_exit** *în registrul EBX (în baza 16) este:* ____

Exerciţiul 7 [1p] *Explicaţi dacă următoarele două secvenţe de cod accesează elementul de pe indexul al doilea al unui array de întregi, stocat în RAM la adresa cu numele* **v**. *Sunt secvenţele de cod echivalente? Dacă da, de ce? Dacă nu, ce accesează fiecare, în parte?*

a. `mov $v, %edi;  movl $2, %ecx;  movl (%edi, %ecx, 4), %ebx`

b. `mov $v, %edi;  addl $8, %edi;  movl $0, %ecx;  movl (%edi, %ecx, 4), %ebx`

Exerciţiul 8 [1p] *Scrieţi o secvenţă de cod în* **main** *care să calculeze suma elementelor dintr-un array, fiecare element fiind înmulţit cu indexul său. Consideraţi că lungimea array-ului este* **n**, *iar numele lui este* **v**, $n \leq 10$, *iar orice element* $v_i \leq 100$. *Aceste informaţii sunt deja în secţiunea* **.data**. *Soluţia voastră trebuie să înceapă de la eticheta* **main**.